ABSTRACT

The Placement Management System is a database-driven web application designed to manage campus recruitment activities in a structured and efficient manner. The system provides a centralized platform where placement administrators can maintain student records, company records, job openings, applications, interview schedules, and placement analytics.

The project is implemented using HTML, CSS, and JavaScript for the frontend, Node.js and Express.js for the backend, and MySQL for relational storage. Authentication is secured using JWT and bcrypt. The schema demonstrates primary keys, foreign keys, unique constraints, normalized tables, and SQL queries for operational and analytical reporting.

The report documents the system background, methodology, requirements, E-R model, relational database design, normalization, data dictionary, graphical interface, source code organization, testing results, conclusion, and future enhancements.

TABLE OF CONTENTS

SL.NO	TITLE	PAGE NUMBER
1	Introduction	1
2	Background, Motivation and Scope	2
3	Methodology	3
4	Requirements	5
5	E-R Diagram	6
6	Relational Database Design	8
7	Database Normalization	13
8	Data Dictionary	15
9	Graphical User Interface	18
10	Source Code	20
11	Test and Results	23
12	Conclusion	27
13	Future Enhancement and Bibliography	28

1. INTRODUCTION

A placement cell manages a large amount of academic, recruitment, and interview data. Manual processing through spreadsheets or paper-based records can create duplicate entries, delayed updates, missing information, and difficulty in preparing analytical reports.

The Placement Management System addresses these issues by storing all placement-related information in a normalized MySQL database and presenting it through a browser-based interface. Administrators can manage students, companies, jobs, applications, interviews, and analytics, while students can view opportunities and track their activity.

Technologies Used

Technology	Purpose
HTML, CSS, JavaScript	Frontend structure, styling, interaction, forms, tables, and dashboard rendering
Node.js and Express.js	Backend runtime, REST API routing, middleware, static file serving, and controllers
MySQL	Relational data storage with keys, constraints, and joins
JWT and bcrypt	Authentication, token-based sessions, and password hashing
Chart.js	Dashboard analytics and visual reporting

2. BACKGROUND, MOTIVATION AND SCOPE

Campus placement activities involve students, recruiters, job roles, eligibility rules, applications, interview rounds, and placement outcomes. A DBMS-based application is suitable because the data is structured, relational, and frequently queried.

The motivation for this project is to reduce manual effort, improve data accuracy, provide secure access, and generate placement statistics quickly. A centralized system allows administrators to monitor recruitment progress and enables students to access relevant placement information.

- The scope includes student, company, job, application, interview, authentication, and dashboard modules.
- The system can be used by a college placement department for academic demonstration and practical record management.
- Future extensions may include company login, resume upload, email notifications, offer letters, and predictive analytics.

3. METHODOLOGY

The project follows a modular full-stack development methodology. The placement workflow was divided into database entities and application modules. The schema was created first, followed by backend APIs, authentication middleware, frontend pages, and dashboard analytics.

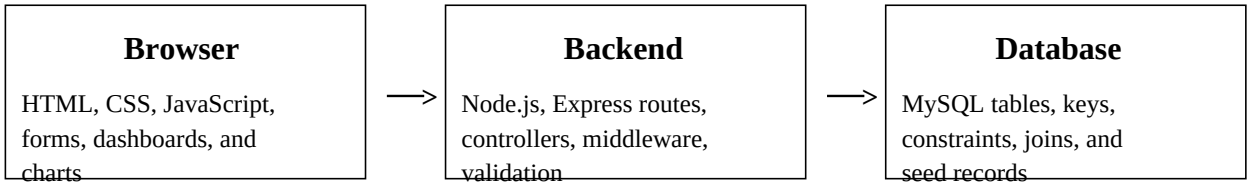


Figure 4.1: System Architecture

3. METHODOLOGY

Development Steps

- Identified user roles, placement workflows, and core entities.
- Designed normalized tables for users, students, companies, jobs, applications, and interviews.
- Created database.sql with primary keys, foreign keys, unique constraints, and seed data.
- Implemented backend route modules and controller functions using Express.js.
- Added JWT authentication, bcrypt password verification, validation, and role-based access control.
- Built frontend pages for login, dashboards, and management modules.
- Connected frontend forms and tables to REST APIs using JavaScript fetch calls.
- Tested login, CRUD operations, application submission, interview scheduling, and dashboard analytics.

4. REQUIREMENTS

Table 5.1: Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Processor	Intel Core i3 or equivalent	Intel Core i5 or better
RAM	4 GB	8 GB or more
Storage	2 GB free space	5 GB free space
Display	HD display	Full HD display
Network	Basic internet access	Stable internet connection

Table 5.2: Software Requirements

Software	Purpose
Node.js	Runs the backend server
MySQL Server	Stores relational project data
Web Browser	Runs the frontend interface
Code Editor	Development and maintenance
npm	Dependency management

Required Packages

Package	Use
express	REST API routing and middleware
mysql2	MySQL connection pool and queries
bcrypt	Password hashing
jsonwebtoken	JWT authentication
cors	Cross-origin API support
dotenv	Environment configuration
express-validator	Request validation

Environment File Example

```
PORT=5000
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=placement_management
JWT_SECRET=change_this_secret
```

5. E-R DIAGRAM

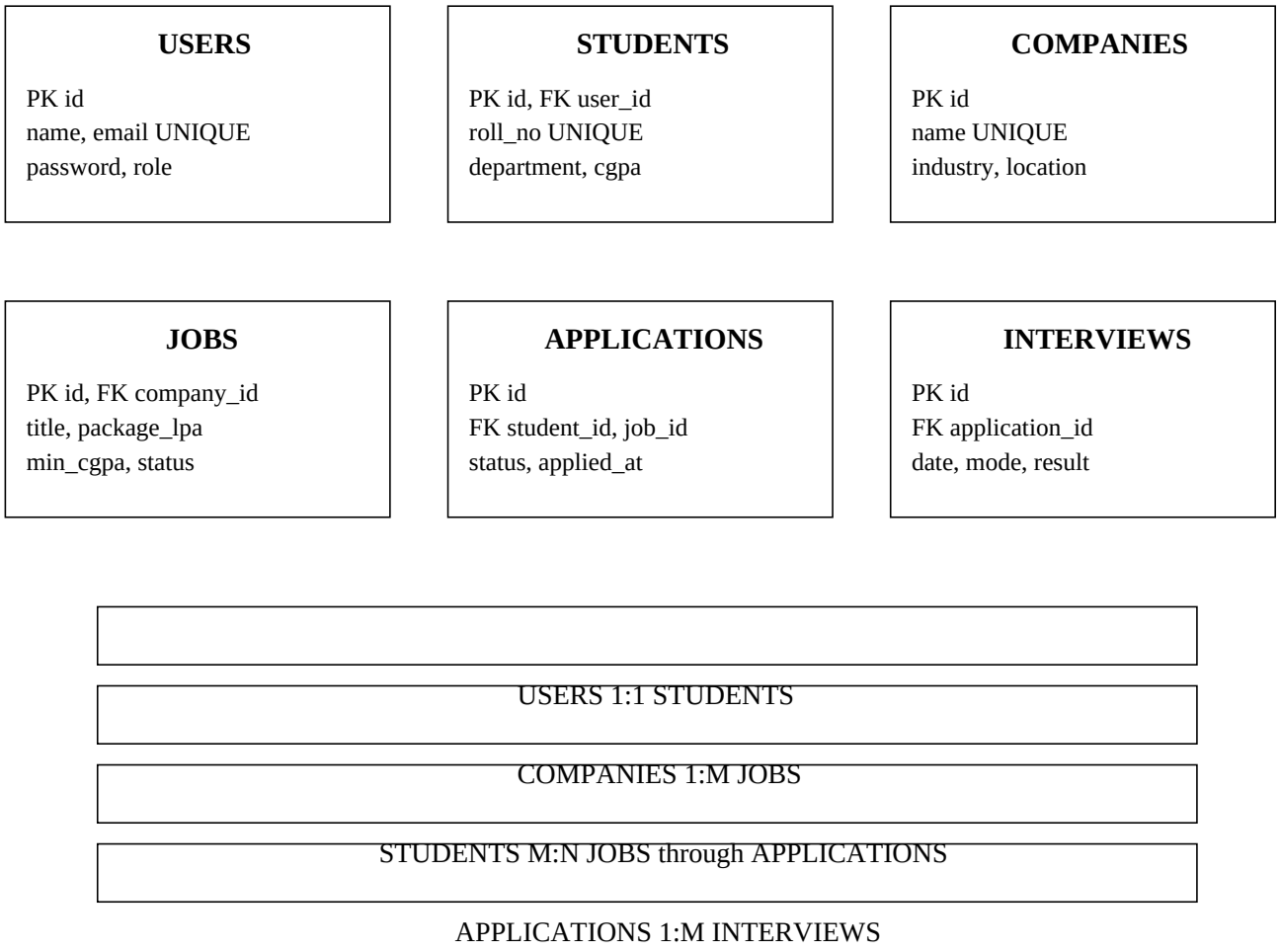


Figure 6.1: E-R Diagram

5. E-R DIAGRAM

The E-R model represents the major entities and relationships in the Placement Management System. The users table stores login credentials and roles. The students table stores academic details and is linked to users. Companies post jobs, students apply for jobs, and interviews are scheduled for applications.

- users to students is a one-to-one relationship because each student account has one profile.
- companies to jobs is a one-to-many relationship because one company can post multiple jobs.
- students to jobs is many-to-many and is resolved through the applications table.
- applications to interviews is one-to-many because an application may have multiple interview rounds.

6. RELATIONAL DATABASE DESIGN

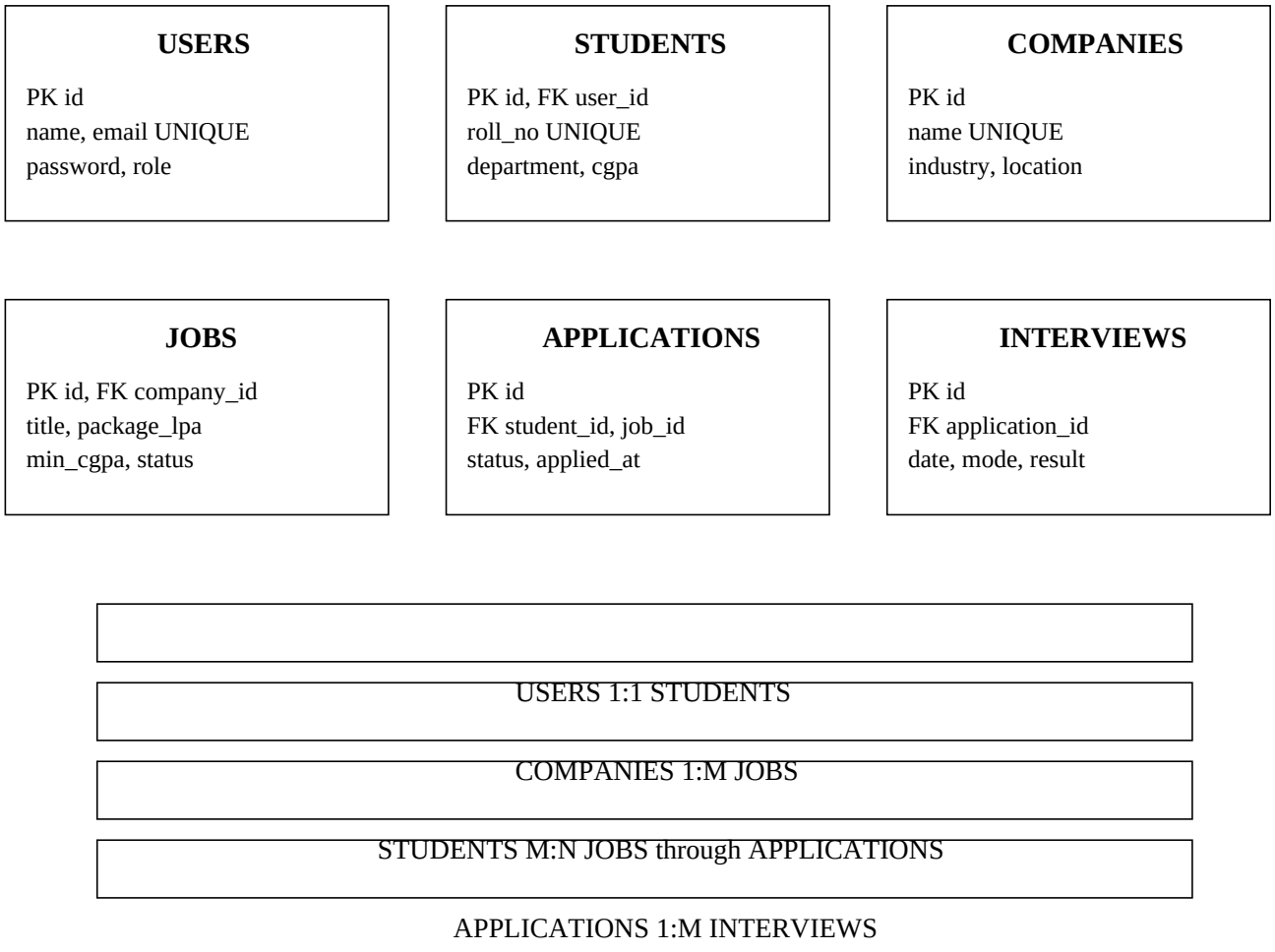


Figure 7.1: Relational Schema Diagram

Relationship Summary: students.user_id references users.id; jobs.company_id references companies.id; applications.student_id references students.id; applications.job_id references jobs.id; interviews.application_id references applications.id.

6. RELATIONAL DATABASE DESIGN

Table 7.1: Users Table

Column	Data Type	Key / Constraint	Purpose
id	INT	PK	User identifier
name	VARCHAR(100)	NOT NULL	Full name
email	VARCHAR(120)	UNIQUE	Login email
password	VARCHAR(255)	NOT NULL	Hashed password
role	ENUM('admin','student')	NOT NULL	Access role
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Creation time

Table 7.2: Students Table

Column	Data Type	Key / Constraint	Purpose
id	INT	PK	Student identifier
user_id	INT	FK, UNIQUE	Linked user account
roll_no	VARCHAR(40)	UNIQUE	Roll number
department	VARCHAR(80)	NOT NULL	Department
graduation_year	YEAR	NOT NULL	Graduation year
cgpa	DECIMAL(3,2)	CHECK	Academic score
skills	TEXT		Technical skills
phone	VARCHAR(20)		Contact number

6. RELATIONAL DATABASE DESIGN

Table 7.3: Companies Table

Column	Data Type	Key / Constraint	Purpose
id	INT	PK	Company identifier
name	VARCHAR(120)	UNIQUE	Company name
industry	VARCHAR(100)		Business domain
website	VARCHAR(180)		Website
location	VARCHAR(120)		Location
contact_email	VARCHAR(120)		HR email

Table 7.4: Jobs Table

Column	Data Type	Key / Constraint	Purpose
id	INT	PK	Job identifier
company_id	INT	FK	Posting company
title	VARCHAR(140)	NOT NULL	Job role
description	TEXT		Responsibilities
package_lpa	DECIMAL(5,2)		Salary package
min_cgpa	DECIMAL(3,2)		Eligibility CGPA
deadline	DATE		Last date
status	ENUM(’open’,’closed’)		Availability

6. RELATIONAL DATABASE DESIGN

Table 7.5: Applications Table

Column	Data Type	Key / Constraint	Purpose
id	INT	PK	Application identifier
student_id	INT	FK	Applicant
job_id	INT	FK	Applied job
status	ENUM('applied','shortlisted','selected','rejected')		Current status
applied_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Submission time
updated_at	TIMESTAMP	ON UPDATE	Status update time
unique_application	UNIQUE	Prevents duplicate applications	

Table 7.6: Interviews Table

Column	Data Type	Key / Constraint	Purpose
id	INT	PK	Interview identifier
application_id	INT	FK	Related application
interview_date	DATETIME	NOT NULL	Date and time
mode	ENUM('online','offline')		Mode
venue	VARCHAR(180)		Venue or meeting link
interviewer	VARCHAR(120)		Interviewer
result	ENUM('scheduled','selected','rejected','on_hold')		Result
notes	TEXT		Remarks

6. RELATIONAL DATABASE DESIGN

The relational design separates authentication, student profiles, companies, job openings, applications, and interviews into independent but connected tables. This design reduces duplication and supports reliable joins for reports and dashboards.

- Primary keys identify each record uniquely.
- Foreign keys preserve relationships between dependent records.
- Unique constraints prevent duplicate accounts, roll numbers, company names, and repeated applications.
- Status fields support controlled workflow tracking for jobs, applications, and interviews.

7. DATABASE NORMALIZATION

First Normal Form (1NF): All fields contain atomic values, and every table has a primary key. For example, student roll number, department, CGPA, and phone number are stored as separate attributes instead of combined text.

Second Normal Form (2NF): Non-key attributes depend on the full key of their table. Job details depend on job id, company details depend on company id, and application status depends on the application record.

The applications table also resolves the many-to-many relationship between students and jobs, preventing repeated groups inside the students or jobs tables.

7. DATABASE NORMALIZATION

Third Normal Form (3NF): Transitive dependencies are removed. Company information is stored in companies and referenced from jobs. Student account credentials are stored in users, while academic profile information is stored in students.

- The design avoids storing company names repeatedly in every job record.
- The design avoids storing student login credentials repeatedly in placement records.
- Applications store only application-related data and reference students and jobs.
- Interviews reference applications, so interview data remains connected to the recruitment process without duplicating student and job details.

8. DATA DICTIONARY

Table 9.1: Users Data Dictionary

Attribute	Data Type	Constraint	Description
id	INT	PK	User identifier
name	VARCHAR(100)	NOT NULL	Full name
email	VARCHAR(120)	UNIQUE	Login email
password	VARCHAR(255)	NOT NULL	Hashed password
role	ENUM('admin','student')	NOT NULL	Access role
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Creation time

Table 9.2: Students Data Dictionary

Attribute	Data Type	Constraint	Description
id	INT	PK	Student identifier
user_id	INT	FK, UNIQUE	Linked user account
roll_no	VARCHAR(40)	UNIQUE	Roll number
department	VARCHAR(80)	NOT NULL	Department
graduation_year	YEAR	NOT NULL	Graduation year
cgpa	DECIMAL(3,2)	CHECK	Academic score
skills	TEXT		Technical skills
phone	VARCHAR(20)		Contact number

8. DATA DICTIONARY

Table 9.3: Companies Data Dictionary

Attribute	Data Type	Constraint	Description
id	INT	PK	Company identifier
name	VARCHAR(120)	UNIQUE	Company name
industry	VARCHAR(100)		Business domain
website	VARCHAR(180)		Website
location	VARCHAR(120)		Location
contact_email	VARCHAR(120)		HR email

Table 9.4: Jobs Data Dictionary

Attribute	Data Type	Constraint	Description
id	INT	PK	Job identifier
company_id	INT	FK	Posting company
title	VARCHAR(140)	NOT NULL	Job role
description	TEXT		Responsibilities
package_lpa	DECIMAL(5,2)		Salary package
min_cgpa	DECIMAL(3,2)		Eligibility CGPA
deadline	DATE		Last date
status	ENUM('open','closed')		Availability

8. DATA DICTIONARY

Table 9.5: Applications Data Dictionary

Attribute	Data Type	Constraint	Description
id	INT	PK	Application identifier
student_id	INT	FK	Applicant
job_id	INT	FK	Applied job
status	ENUM(’applied’,’shortlisted’,’selected’,’rejected’)		Current status
applied_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Submission time
updated_at	TIMESTAMP	ON UPDATE	Status update time
unique_application	UNIQUE	Prevents duplicate applications	undefined

Table 9.6: Interviews Data Dictionary

Attribute	Data Type	Constraint	Description
id	INT	PK	Interview identifier
application_id	INT	FK	Related application
interview_date	DATETIME	NOT NULL	Date and time
mode	ENUM(’online’,’offline’)		Mode
venue	VARCHAR(180)		Venue or meeting link
interviewer	VARCHAR(120)		Interviewer
result	ENUM(’scheduled’,’selected’,’rejected’,’on_hold’)		Result
notes	TEXT		Remarks

9. GRAPHICAL USER INTERFACE

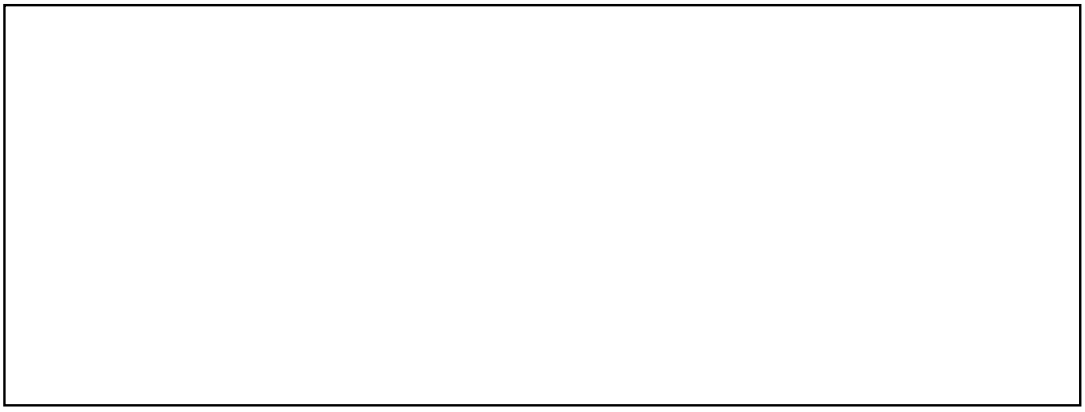


Figure 10.1: Login Page

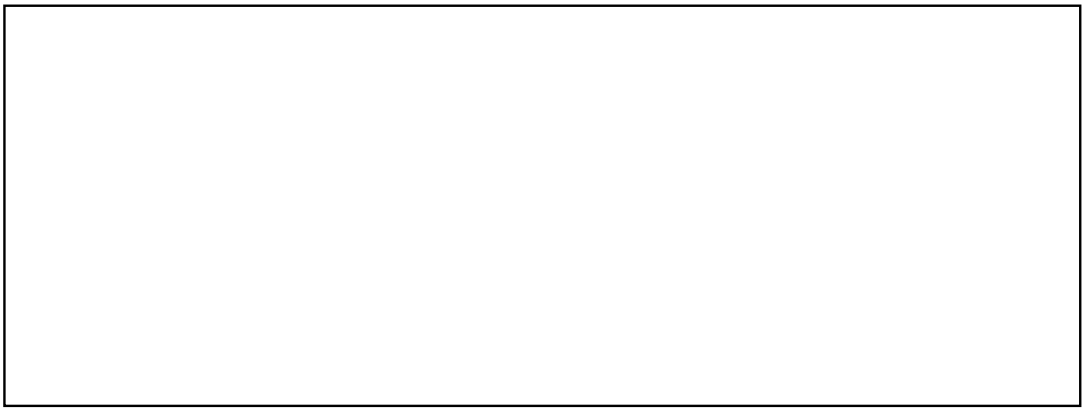


Figure 10.2: Admin Dashboard

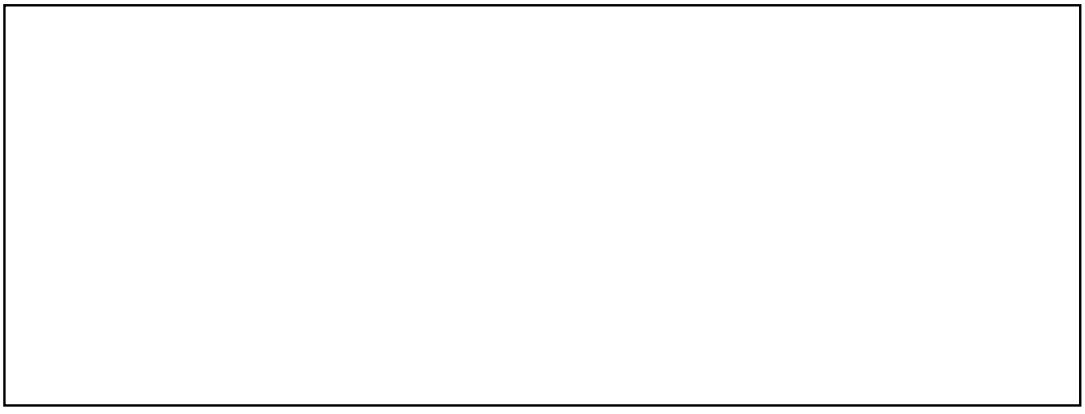


Figure 10.3: Students Page

9. GRAPHICAL USER INTERFACE

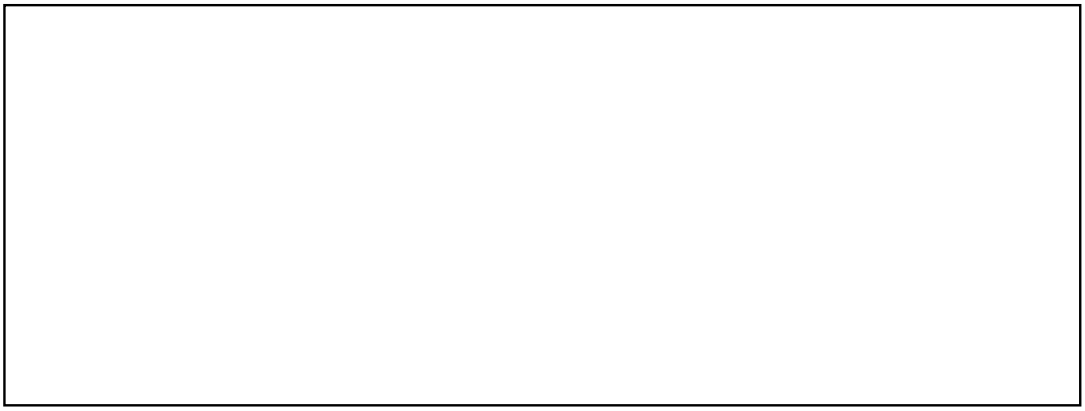


Figure 10.4: Jobs Page

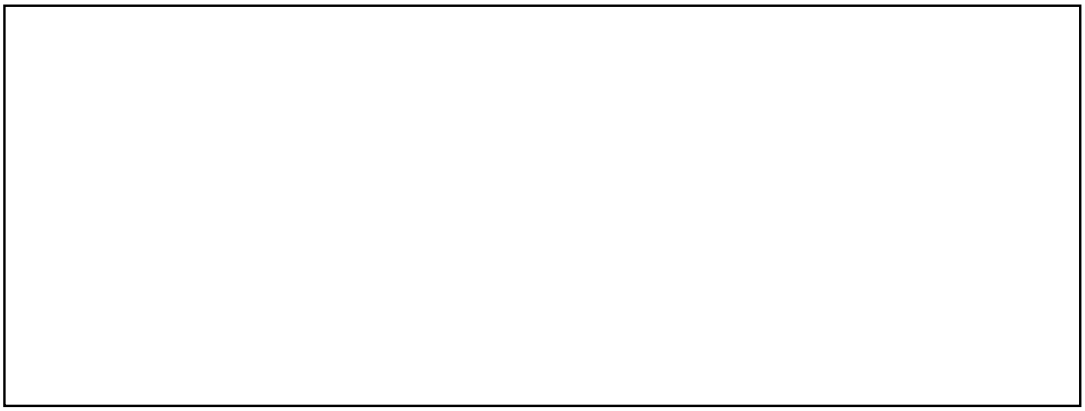


Figure 10.5: Applications Page

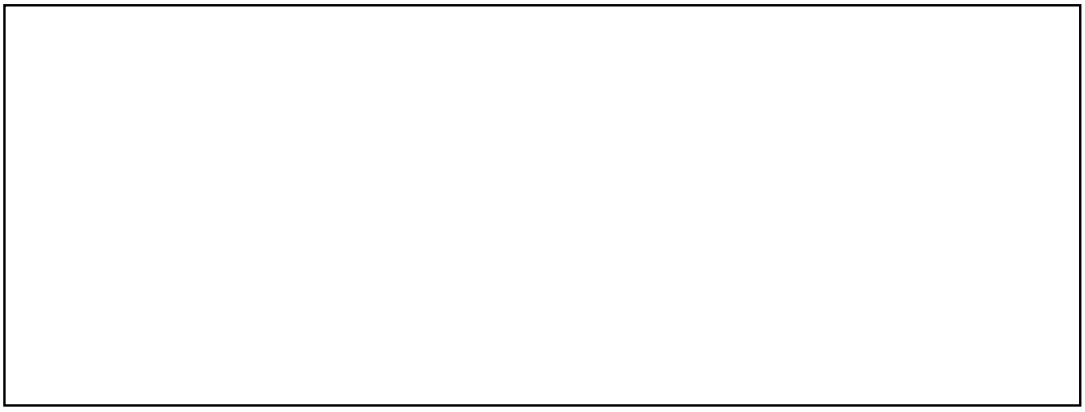


Figure 10.6: Interviews Page

10. SOURCE CODE

Table 11.1: Frontend File Structure

File	Description
frontend/index.html	Login page for admin and student access
frontend/admin-dashboard.html	Admin dashboard and analytics interface
frontend/student-dashboard.html	Student profile, jobs, and applications view
frontend/students.html	Student management page
frontend/jobs.html	Job management page
frontend/applications.html	Application status management page
frontend/interviews.html	Interview scheduling and result page
frontend/assets/js	API calls, validation, rendering, and charts

10. SOURCE CODE

Table 11.2: Backend File Structure

File / Folder	Description
backend/server.js	Configures middleware, static files, routes, and server startup
backend/src/config/db.js	Creates MySQL connection pool
backend/src/controllers	Contains module business logic
backend/src/routes	Defines REST API endpoints
backend/src/middleware/authMiddleware.js	Verifies JWT and role permissions
backend/src/middleware/validate.js	Runs request validation
database.sql	Creates database schema and seed records

10. SOURCE CODE

Key Code Snippets

```
app.use(express.json());

app.use('/api/auth', authRoutes);

app.use('/api/students', studentRoutes);

const token = jwt.sign({ id: user.id, role: user.role }, JWT_SECRET);

const [rows] = await pool.query('SELECT * FROM users WHERE email = ? AND role = ?', [email, role]);

headers.Authorization = `Bearer ${token}`;
```

These snippets show server middleware setup, route mounting, JWT token generation, secure parameterized SQL queries, and frontend token-based API access.

11. TEST AND RESULTS

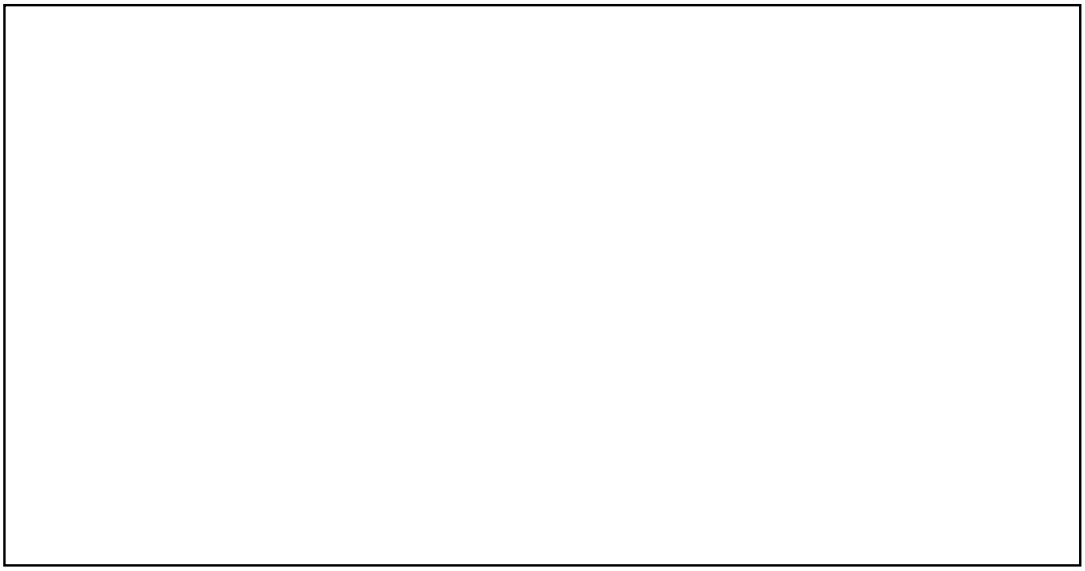


Figure 12.1: Analytics Page

The analytics page verifies that dashboard cards and charts load database values such as total students, selected students, placement percentage, highest package, and average package.

11. TEST AND RESULTS

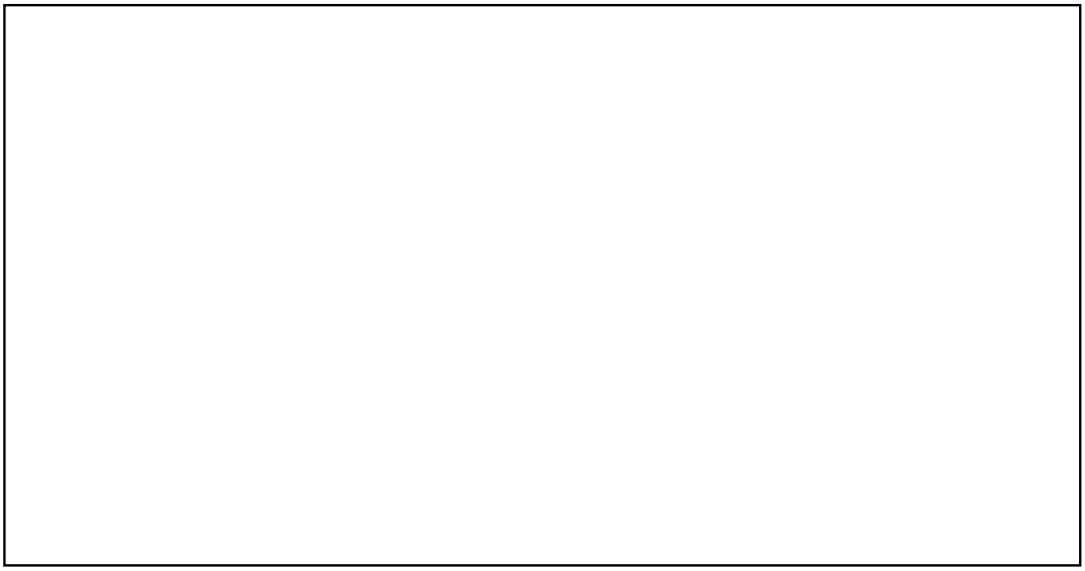


Figure 12.2: Students Module Output

The students module verifies that records can be viewed, added, edited, and deleted by an administrator while maintaining account and profile consistency.

11. TEST AND RESULTS

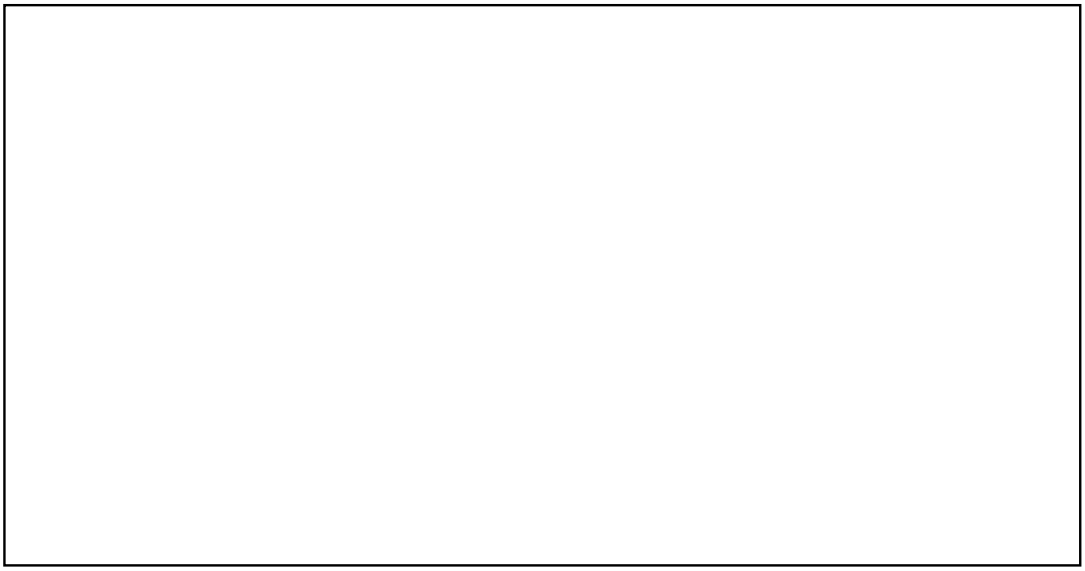


Figure 12.3: Interview Module Output

The interview module verifies scheduling, interview mode, venue, interviewer, result, and notes for each selected application.

11. TEST AND RESULTS

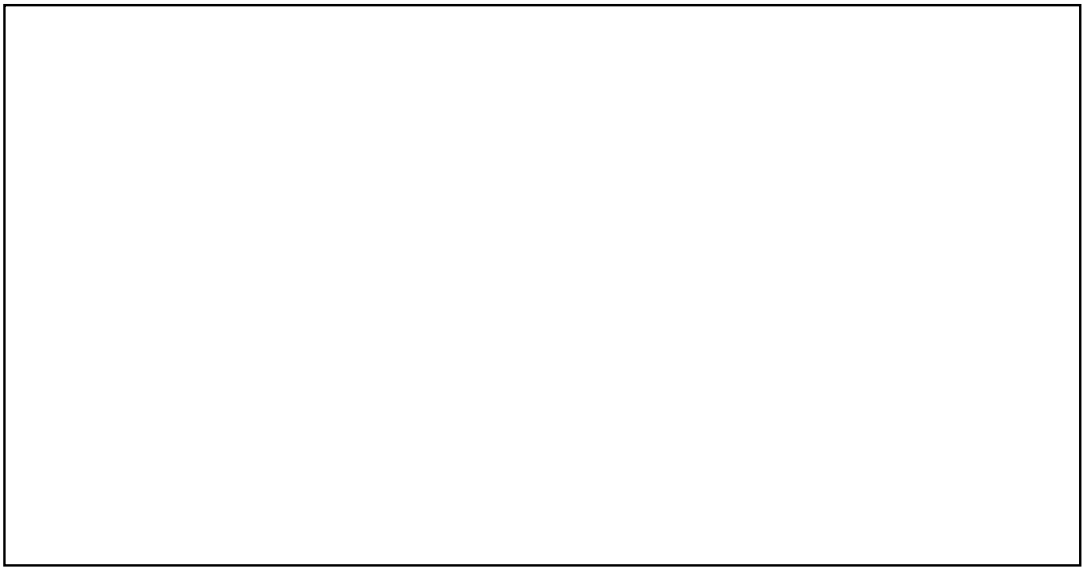


Figure 12.4: Jobs Module Output

The jobs module verifies job creation, company linkage, eligibility CGPA, package details, deadline, status, and listing for students.

12. CONCLUSION

The Placement Management System successfully demonstrates the application of relational database concepts to a real academic workflow. The system uses normalized tables, primary keys, foreign keys, unique constraints, transactions, joins, aggregate queries, secure authentication, and role-based access control.

The project improves placement management by centralizing records, reducing duplicate data, enabling secure access, and providing useful analytics for placement administrators. It also provides students with a structured way to view jobs, apply for opportunities, and track interviews.

13. FUTURE ENHANCEMENT AND BIBLIOGRAPHY

Future enhancements may include company login, resume upload storage, email and SMS notifications, interview round tracking, offer letter generation, PDF or Excel export, advanced eligibility filtering, and predictive analytics for placement trends.

Bibliography

- MySQL Documentation, Relational Database Concepts and SQL Syntax.
- Node.js Documentation, Server-Side JavaScript Runtime.
- Express.js Documentation, Routing and Middleware.
- JSON Web Token Documentation, Token-Based Authentication.
- bcrypt Documentation, Password Hashing.
- Chart.js Documentation, Browser-Based Data Visualization.
- Project files: backend/server.js, backend/src, frontend, and database.sql.